

The University of Lahore, CS & IT Department

Object Oriented Programming

Lab Assignment # 01

Start Date: 26/10/2024

Total Marks: 20

Due Date: 10/11/2024

Program: BSCS

Instructions

1. Understanding the problems is part of the assignment. So, no query, please.
 2. You will get zero marks if found any type of plagiarism or AI generated code.
 3. No submission after due date.
 4. Upload pdf file containing your code and screenshots of outputs.
-

Q1: Create a class-based system to estimate how many deliveries the drone can complete before its battery is drained.

Task Details:

1. Create a class DroneDeliveryEstimator with the following private attributes:
 - a. batteryLevel (float): Represents the current battery level of the drone (initially set to 100%).
 - b. deliveryCount (int): Keeps track of the number of deliveries the drone has completed.
2. Define the following public member functions:
 - a. MakeDelivery(int minutes): Reduces the battery level based on the delivery time in minutes. The drone consumes 1.5% of the battery per minute. If the drone does not have enough battery to complete the next delivery, stop the task and display a message indicating that the drone can't perform more deliveries.
 - b. GetRemainingBattery() const: A constant member function that returns the remaining battery percentage after all deliveries are completed.
 - c. GetCompletedDeliveries() const: A constant member function that returns the total number of deliveries completed.
3. In the main() function:
 - a. Create an object drone of the DroneDeliveryEstimator class.
 - b. Use a do-while loop to ask for the estimated flight time (in minutes) for each delivery.
 - c. After each delivery, check if the drone has enough battery to perform the next delivery. If not, stop the loop.
 - d. Display the total number of deliveries completed and the remaining battery percentage at the end.

Sample Output:

Enter time for delivery in minutes: 30

Do you want to enter another delivery time? (y/n): y

Enter time for delivery in minutes: 20

Do you want to enter another delivery time? (y/n): y

Enter time for delivery in minutes: 15

Do you want to enter another delivery time? (y/n): y

Enter time for delivery in minutes: 25

Drone doesn't have enough battery to perform the last entered delivery.

According to your estimated time, the drone can complete 3 delivery(ies), and the remaining battery will be 10.5%.

Q2: Mrs. Sarah has a keen eye for art and enjoys decorating her room with unique paintings. However, due to the limited wall space, she must carefully manage the weight of the paintings she hangs to avoid overloading the wall. The wall can support a total weight of 10,000 grams, and Sarah wants a reliable system to help her track the paintings she adds, monitor their total weight, and prevent adding new paintings if they exceed the wall's weight limit.

Task Details:

1. Create a class `Painting` with the following private attributes:
 - a. `paintingID (int)`: A unique ID for each painting.
 - b. `title (string)`: The title of the painting.
 - c. `weight (int)`: The weight of the painting in grams.
 - d. Static data member `totalPaintings (int)`: To keep track of the total number of paintings added to the wall.
 - e. Static data member `weightCapacity (int)`: To track the remaining wall capacity, initialized to 10,000 grams.
2. Implement the following:
 - a. Parameterized constructor: Initializes `paintingID`, `title`, and `weight`, increments `totalPaintings` by 1, and decreases `weightCapacity` by the weight of each new painting.
 - b. Static member function `getTotalPaintings()`: Public function, returns the total number of paintings added.
 - c. Static member function `getWeightCapacity()`: Public function, returns the remaining wall capacity.
3. In the `main()` function:
 - a. Use a do-while loop to allow Mrs. Sarah to add multiple paintings to the gallery wall. Ask for the `paintingID`, `title`, and `weight` for each painting.
 - b. Before creating a painting object, check if the weight entered is less than or equal to the current `weightCapacity`. If yes, create the object; otherwise, display a message that the painting cannot be added due to insufficient wall capacity.
 - c. At the end: Display the total number of paintings added and the remaining wall capacity using the static member functions.

Sample Output:

```
Enter Painting ID: 201
Enter Painting Title: Sunset Beauty
Enter Weight of Painting in grams: 1500
Painting added successfully!
Do you want to add another painting? (y/n): y
Enter Painting ID: 202
Enter Painting Title: Ocean Waves
Enter Weight of Painting in grams: 2500
Painting added successfully!
Do you want to add another painting? (y/n): y
Enter Painting ID: 203
Enter Painting Title: Mountain Serenity
Enter Weight of Painting in grams: 7000
Painting cannot be added! Insufficient wall capacity.
Do you want to add another painting? (y/n): n

Total Paintings in Gallery: 2
Remaining Wall Capacity: 6000 grams
```